```
import numpy as np
from scipy.optimize import minimize
import csv
import traceback
import matplotlib.pyplot as plt
from matplotlib.colors import Normalize
from tqdm import tqdm
import os

# ============================================================
# 1. PHYSICAL CONSTANTS (same as V19)
# ============================================================
e = 1.602176634e-19
m_p = 1.67262192369e-27
m_n = 1.67492749804e-27
c = 299792458.0
h = 6.62607015e-34
hbar = h / (2 * np.pi)
mu0 = 4e-7 * np.pi
epsilon0 = 1 / (mu0 * c**2)
MeV = 1.602176634e-13
fm = 1e-15

# ============================================================
# 2. NEUTRON GEOMETRY (unchanged)
# ============================================================
R_plus = 0.8415 * fm
R_minus = 0.478 * fm
r0_physical = 2 * R_plus
```

```
# ============================================================
# 3. LOCAL FIELD STRENGTH & COHERENCE (unchanged)
# ============================================================
def local_field_strength(r, coupling_order=3):
    if r == 0:
        return 1e6
    r_scaled = r / fm
    return 1.0 / (r_scaled ** coupling_order + 1e-30)

def coherence_fraction(E, eta_0, E0, Emax=None):
    if Emax is None or Emax == np.inf:
        return 1 - (1 - eta_0) * np.exp(-E / E0)
    else:
        base = 1 - (1 - eta_0) * np.exp(-E / E0)
        roll_off = np.exp(-(E / Emax) ** 2)
        return base * roll_off

# ============================================================
# 4. INDUCTANCE & COULOMB FUNCTIONS (unchanged)
# ============================================================
def generate_loop_points(center, tilt, yaw, radius=0.8415e-15, steps=12):
    t = np.linspace(0, 2 * np.pi, steps, endpoint=False)
    base_points = np.zeros((steps, 3))
    base_points[:, 0] = radius * np.cos(t)
    base_points[:, 1] = radius * np.sin(t)
    cos_t, sin_t = np.cos(tilt), np.sin(tilt)
    cos_y, sin_y = np.cos(yaw), np.sin(yaw)
    R_tilt = np.array([[1.0, 0.0, 0.0], [0.0, cos_t, -sin_t], [0.0, sin_t, cos_t]])
    R_yaw = np.array([[cos_y, -sin_y, 0.0], [sin_y, cos_y, 0.0], [0.0, 0.0, 1.0]])
    return np.dot(base_points, np.dot(R_yaw, R_tilt).T) + center
```

```
def calculate_mutual_inductance(loop1, loop2):
    n = len(loop1)
    dl1 = np.zeros((n, 3))
    dl1[:-1] = loop1[1:] - loop1[:-1]
    dl1[-1] = loop1[0] - loop1[-1]
    dl2 = np.zeros((n, 3))
    dl2[:-1] = loop2[1:] - loop2[:-1]
    dl2[-1] = loop2[0] - loop2[-1]
    diff = loop1[:, np.newaxis, :] - loop2[np.newaxis, :, :]
    r12 = np.linalg.norm(diff, axis=2)
    core_buffer = 0.1e-15
    r12 = np.where(r12 < core_buffer, core_buffer, r12)
    M = -(mu0 / (4 * np.pi)) * np.sum(np.sum(dl1[:, np.newaxis, :] * dl2[np.newaxis, :, :], axis=2) / r12)
    return M

def calculate_self_inductance(loop, radius, q, I):
    a_wire = 0.1e-15
    L = mu0 * radius * (np.log(8 * radius / a_wire) - 2)
    return L

def calculate_coulomb_energy(q1, q2, r):
    if r == 0:
        return 1e6
    return (1 / (4 * np.pi * epsilon0)) * q1 * q2 / r

# ============================================================
# 5. GENERALISED GEOMETRY GENERATOR (with compaction)
# ============================================================
def generate_nucleus_geometry(Z, N, seed=42, compaction=0.9):
```

```
    A = Z + N
    R0 = compaction * 1.2 * (A ** (1.0/3.0)) * fm
    np.random.seed(seed)
    positions = []
    identities = []
    for i in range(A):
        theta = np.arccos(2 * np.random.rand() - 1)
        phi = 2 * np.pi * np.random.rand()
        r = R0 * np.random.rand() ** (1.0/3.0)
        pos = np.array([r * np.sin(theta) * np.cos(phi),
                        r * np.sin(theta) * np.sin(phi),
                        r * np.cos(theta)])
        positions.append(pos)
        identities.append(0 if i < Z else 1)
    positions = np.array(positions)
    identities = np.array(identities)
    com = np.mean(positions, axis=0)
    positions -= com
    return positions, identities

# ============================================================
# 6. NUCLEON SOLVER V19 (PATCHED: dynamic bounds)
# ============================================================
class NucleonSolverV19:
    def __init__(self, positions, identities, parameters):
        self.centers_initial = positions
        self.identities = identities
        self.n_loops = len(positions)
        self.n_params = 5 * self.n_loops
        self.Z = np.sum(identities == 0)
```

```
    self.N = np.sum(identities == 1)
    self.I_p = e * (m_p * c**2 / h)
    self.I_n_base = e * (m_n * c**2 / h)
    self.repulsion_strength = parameters['repulsion_strength']
    self.neutron_eta_0 = 0.676
    self.neutron_E0 = parameters['neutron_E0']
    self.proton_eta_0 = 1.0
    self.proton_E0 = 1.5
    self.Emax = parameters['Emax']
    self.alpha_scale = parameters.get('alpha_scale', 1.0)
    self.r0 = r0_physical

    self.U0 = None
    self.U_min = None
    self.delta_E = None
    self.free_energy = None
    self.final_centers = None
    self.movement_distance = None
    self.mag_attractive_energy = None
    self.repulsive_energy = None
    self.boundary_hit = False

def compute_eta_for_nucleon(self, idx, centers):
    center = centers[idx]
    E_total = 0.0
    for j, other in enumerate(centers):
        if j == idx:
            continue
        r = np.linalg.norm(center - other)
        E_total += local_field_strength(r, coupling_order=3)
```

```
    if self.identities[idx] == 0:
        return coherence_fraction(E_total, self.proton_eta_0, self.proton_E0, self.Emax)
    else:
        return coherence_fraction(E_total, self.neutron_eta_0, self.neutron_E0, self.Emax)


def compute_currents(self, centers):
    currents = []
    for idx, identity in enumerate(self.identities):
        eta = self.compute_eta_for_nucleon(idx, centers)
        if identity == 0:
            currents.append(self.I_p * eta)
        else:
            currents.append(self.I_n_base * eta)
    return currents


def pack_params(self, centers, angles):
    params = np.zeros(5 * self.n_loops)
    for i in range(self.n_loops):
        params[5*i : 5*i + 3] = centers[i] / fm
        params[5*i + 3 : 5*i + 5] = angles[i]
    return params


def unpack_params(self, params):
    centers = np.zeros((self.n_loops, 3))
    angles = np.zeros((self.n_loops, 2))
    for i in range(self.n_loops):
        centers[i] = params[5*i : 5*i + 3] * fm
        angles[i] = params[5*i + 3 : 5*i + 5]
    return centers, angles
```

```
def calculate_energy_components(self, params):
    centers, angles = self.unpack_params(params)
    loops = []
    for i in range(self.n_loops):
        loops.append(generate_loop_points(centers[i], angles[i, 0], angles[i, 1]))
    currents = self.compute_currents(centers)
    charges = [e if self.identities[i] == 0 else 0.0 for i in range(self.n_loops)]

    total_energy_joules = 0.0
    mag_attractive_joules = 0.0
    repulsive_joules = 0.0
    radius = 0.8415e-15

    mag_joules = 0.0
    for i in range(self.n_loops):
        L_ii = calculate_self_inductance(loops[i], radius, charges[i], currents[i])
        mag_joules -= 0.5 * L_ii * currents[i]**2
    for i in range(self.n_loops):
        for j in range(i + 1, self.n_loops):
            M_ij = calculate_mutual_inductance(loops[i], loops[j])
            mag_joules += M_ij * currents[i] * currents[j]
    mag_joules_scaled = self.alpha_scale * mag_joules
    mag_attractive_joules += mag_joules_scaled

    coulomb_joules = 0.0
    for i in range(self.n_loops):
        for j in range(i + 1, self.n_loops):
            if self.identities[i] == 0 and self.identities[j] == 0:
                r = np.linalg.norm(centers[i] - centers[j])
                coulomb_joules += calculate_coulomb_energy(e, e, r)
```

```
    repulsive_joules += coulomb_joules

    kinetic_joules = 0.0
    for i in range(self.n_loops):
        if self.identities[i] == 0:
            kinetic_joules += 0.5 * m_p * c**2
        else:
            kinetic_joules += 0.5 * m_n * c**2

    core_rep_joules = 0.0
    for i in range(self.n_loops):
        for j in range(i + 1, self.n_loops):
            r = np.linalg.norm(centers[i] - centers[j])
            if r > 0:
                core_rep_joules += self.repulsion_strength * MeV * np.exp(-(r / self.r0) ** 2)
    repulsive_joules += core_rep_joules

    total_energy_joules = mag_joules_scaled + coulomb_joules + kinetic_joules + core_rep_joules

    return (total_energy_joules / MeV,
            mag_attractive_joules / MeV,
            repulsive_joules / MeV)

def calculate_energy(self, params):
    total_mev, _, _ = self.calculate_energy_components(params)
    return total_mev

def calculate_free_nucleon_energy(self):
    total_energy_joules = 0.0
    for i in range(self.n_loops):
```

```
        if self.identities[i] == 0:
            total_energy_joules += 0.5 * m_p * c**2
        else:
            total_energy_joules += 0.5 * m_n * c**2
    return total_energy_joules / MeV

def solve(self, verbose=False, half_range=None):
    if half_range is None:
        nuclear_radius = 1.2 * (self.n_loops ** (1.0/3.0))
        half_range = max(5.0, 2.5 * nuclear_radius)

    initial_angles = np.zeros((self.n_loops, 2))
    initial_params = self.pack_params(self.centers_initial, initial_angles)

    self.U0 = self.calculate_energy(initial_params)

    angle_range = np.pi / 2
    bounds = []
    for i in range(self.n_loops):
        bounds.extend([
            (self.centers_initial[i, 0]/fm - half_range, self.centers_initial[i, 0]/fm + half_range),
            (self.centers_initial[i, 1]/fm - half_range, self.centers_initial[i, 1]/fm + half_range),
            (self.centers_initial[i, 2]/fm - half_range, self.centers_initial[i, 2]/fm + half_range)
        ])
        bounds.extend([
            (-angle_range, angle_range),
            (-angle_range, angle_range)
        ])

    np.random.seed(42)
```

```
initial_params += np.random.uniform(-0.01, 0.01, self.n_params)

result = minimize(
    self.calculate_energy,
    initial_params,
    method='L-BFGS-B',
    bounds=bounds,
    options={'maxiter': 1000, 'ftol': 1e-8}
)

self.U_min = result.fun
self.free_energy = self.calculate_free_nucleon_energy()
self.delta_E = self.free_energy - self.U_min

self.final_centers, self.final_angles = self.unpack_params(result.x)
distances = np.linalg.norm(self.final_centers - self.centers_initial, axis=1)
self.movement_distance = np.mean(distances) / fm

_, mag_att, rep = self.calculate_energy_components(result.x)
self.mag_attractive_energy = mag_att
self.repulsive_energy = rep

tolerance = 0.01 * half_range
hit = False
for i in range(self.n_loops):
    for axis in range(3):
        lower = self.centers_initial[i, axis]/fm - half_range
        upper = self.centers_initial[i, axis]/fm + half_range
        val = result.x[5*i + axis]
        if (val - lower < tolerance) or (upper - val < tolerance):
```

```
                hit = True
                break
            if hit:
                break
        self.boundary_hit = hit

        return result

# ============================================================
# 7. SCAN FUNCTION V1.1 (with append mode, saves every iteration)
# ============================================================
def run_stability_scan_v1_1(Z_min=1, Z_max=20, N_ratio_min=1.0, N_ratio_max=3.0,
                            alpha_scale=1.0, repulsion=0.5, Emax=0.5, neutron_E0=0.5,
                            compaction=0.9, verbose=True, output_file='stability_scan_results_v1.1.csv'):
    """
    V1.1: Saves results immediately after each nuclide using append mode.
    No data loss risk—every nuclide is written to disk instantly.
    """
    # Count total combinations
    total_combinations = 0
    for Z in range(Z_min, Z_max+1):
        for N in range(int(np.ceil(Z*N_ratio_min)), int(np.floor(Z*N_ratio_max))+1):
            total_combinations += 1

    # Write header (overwrites any existing file)
    with open(output_file, 'w', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=['Z', 'N', 'A', 'delta_E', 'movement', 'stable', 'boundary_hit'])
        writer.writeheader()

    # Progress bar
```

```
    progress = tqdm(total=total_combinations, desc="Scanning nuclides") if verbose else None
    idx = 0
    results = []  # Still keep in memory for heatmap later

    for Z in range(Z_min, Z_max+1):
        for N in range(int(np.ceil(Z*N_ratio_min)), int(np.floor(Z*N_ratio_max))+1):
            idx += 1

            parameters = {
                'alpha_scale': alpha_scale,
                'repulsion_strength': repulsion,
                'neutron_E0': neutron_E0,
                'Emax': Emax,
            }

            try:
                positions, identities = generate_nucleus_geometry(Z, N, seed=42+idx,
compaction=compaction)
                solver = NucleonSolverV19(positions, identities, parameters)
                solver.solve(verbose=False)
                delta_E = solver.delta_E
                movement = solver.movement_distance
                stable = delta_E > 0
                boundary_hit = solver.boundary_hit
            except Exception as e:
                delta_E = np.nan
                movement = np.nan
                stable = False
                boundary_hit = False
                if verbose:
```

```
                print(f"  Error on Z={Z}, N={N}: {e}")

        result = {
            'Z': Z, 'N': N, 'A': Z+N,
            'delta_E': delta_E,
            'movement': movement,
            'stable': stable,
            'boundary_hit': boundary_hit,
        }
        results.append(result)

        # 🔥 Append this single result to the CSV immediately
        with open(output_file, 'a', newline='') as f:
            writer = csv.DictWriter(f, fieldnames=['Z', 'N', 'A', 'delta_E', 'movement', 'stable',
'boundary_hit'])
            writer.writerow(result)

        if progress:
            progress.update(1)

    if progress:
        progress.close()

    print(f"\n✅ Scan complete. Total nuclides: {len(results)}")
    print(f"   Results saved to '{output_file}'")
    return results

# ============================================================
# 8. PLOTTING FUNCTIONS (optional)
# ============================================================
```

```
def plot_heatmap(results, save_file='stability_heatmap_v1.1.png'):
    Z_vals = sorted(set(r['Z'] for r in results))
    N_vals = sorted(set(r['N'] for r in results))
    delta_grid = np.full((len(Z_vals), len(N_vals)), np.nan)
    for r in results:
        i = Z_vals.index(r['Z'])
        j = N_vals.index(r['N'])
        delta_grid[i, j] = r['delta_E']

    fig, ax = plt.subplots(figsize=(12, 8))
    im = ax.imshow(delta_grid, origin='lower', aspect='auto',
                   extent=[min(N_vals)-0.5, max(N_vals)+0.5,
                           min(Z_vals)-0.5, max(Z_vals)+0.5],
                   cmap='RdYlBu_r', norm=Normalize(vmin=-5, vmax=30))
    ax.set_xlabel('Neutron number N')
    ax.set_ylabel('Proton number Z')
    ax.set_title('Binding Energy ΔE (MeV) for RealQM V1.1 Stability Scan')
    cbar = plt.colorbar(im, ax=ax)
    cbar.set_label('ΔE (MeV)')

    stable_points = [(r['N'], r['Z']) for r in results if r['stable']]
    if stable_points:
        Ns, Zs = zip(*stable_points)
        ax.scatter(Ns, Zs, s=30, facecolors='none', edgecolors='lime', linewidths=1.5,
                   label='Predicted stable')

    N_line = np.arange(min(N_vals), max(N_vals)+1)
    Z_line_stable = N_line * 1.0
    ax.plot(N_line, Z_line_stable, 'k--', linewidth=0.8, alpha=0.5, label='N=Z')
```

```
    ax.legend()
    plt.tight_layout()
    plt.savefig(save_file, dpi=150)
    print(f"Heatmap saved as '{save_file}'")

def list_stable_isotopes(results):
    stable = [r for r in results if r['stable']]
    stable_sorted = sorted(stable, key=lambda x: (x['Z'], x['N']))
    print("\nPredicted Stable Isotopes (ΔE > 0):")
    print("  Z   N   A   ΔE (MeV)   boundary_hit?")
    for r in stable_sorted:
        hit = "YES" if r['boundary_hit'] else "no"
        print(f" {r['Z']:2d}  {r['N']:2d}  {r['A']:2d}   {r['delta_E']:6.2f}      {hit}")
    return stable_sorted

# ============================================================
# 9. MAIN EXECUTION
# ============================================================
if __name__ == "__main__":
    print("=" * 80)
    print("RealQM Nuclear Stability Scanner V1.1 (Instant Saving)")
    print("=" * 80)
    print("Using calibrated parameters:")
    print("  alpha_scale = 1.00")
    print("  repulsion_strength = 0.50 MeV")
    print("  Emax = 0.5")
    print("  neutron_E0 = 0.5")
    print("  Dynamic half_range = max(5.0, 2.5 * 1.2*A^(1/3)) fm")
    print("  Initial geometry compaction = 0.9")
    print("  Save mode: Append after each nuclide (instant checkpointing)")
```

```
print("Scanning Z = 1 to 20, N = Z to 3Z")
print("=" * 80)

# Run the scan
results = run_stability_scan_v1_1(
    Z_min=1, Z_max=20,
    N_ratio_min=1.0, N_ratio_max=3.0,
    alpha_scale=1.0,
    repulsion=0.5,
    Emax=0.5,
    neutron_E0=0.5,
    compaction=0.9,
    verbose=True,
    output_file='stability_scan_results_v1.1.csv'
)

# Generate heatmap
plot_heatmap(results)

# List stable isotopes
stable_list = list_stable_isotopes(results)

# Summary
total_hits = sum(1 for r in results if r['boundary_hit'])
print(f"\nBoundary hit warnings: {total_hits} out of {len(results)} nuclides.")
if total_hits > 0:
    print("Check the CSV for specific cases where 'boundary_hit' is True.")

print("\nScan complete.")
```